

PoC

PoC - DepthFirstDisclosures, GitHub.

- Published: date not stated
- Original: <<https://github.com/DepthFirstDisclosures/Nginx-Rift>>
- Preserved from: <https://github.com/DepthFirstDisclosures/Nginx-Rift> (github-repository-api) on 2026-10-02
- Repository commit: b9045c32af035d2d6638c29e4822c7da0aab5ade
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from documentation in [DepthFirstDisclosures/Nginx-Rift](#), pinned to commit [b9045c32af03](#). GitHub navigation and file listings are omitted. This is selected documentation; repository code is never checked out, built or run.

README.md

[View original document](#)

NGINX Rift

RCE Proof of concept for **CVE-2026-42945**, a critical heap buffer overflow in NGINX's `ngx_http_rewrite_module` introduced in 2008. The bug enables unauthenticated remote code execution against servers using `rewrite` and `set` directives.

This vulnerability — along with three other memory corruption issues (CVE-2026-42946, CVE-2026-40701, CVE-2026-42934) — was autonomously discovered by [depthfirst](#)'s security analysis system after a single click of onboarding the NGINX source.

Want to find issues like this in your own code? Try the same system at [<https://depthfirst.com/open-defense>](https://depthfirst.com/open-defense).

The Bug (TL;DR)

NGINX's script engine uses a two-pass process: first compute the required buffer size, then copy data in. The `is_args` flag is set on the main engine when a `rewrite` replacement

contains `?`, but the length-calculation pass runs on a freshly zeroed sub-engine. So:

- **Length pass** sees `is_args = 0` → returns raw capture length.
- **Copy pass** sees `is_args = 1` → calls `ngx_escape_uri` with `NGX_ESCAPE_ARGS`, expanding each escapable byte to 3 bytes.

The copy overflows the undersized heap buffer with attacker-controlled URI data. Exploitation uses cross-request heap feng shui to corrupt an adjacent `ngx_pool_t`'s `cleanup` pointer (sprayed via POST bodies, since URI bytes can't contain null bytes), redirecting it to a fake `ngx_pool_cleanup_s` invoking `system()` on pool destruction. Read more about this bug in our [technical write-up](#).

Affected & Fixed Versions

Product	Affected	Fixed in
NGINX Open Source	0.6.27 – 1.30.0	1.31.0, 1.30.1
NGINX Plus	R32 – R36	R36 P4, R35 P2, R32 P6

Full vendor advisory: <<https://my.f5.com/manage/s/article/K000160932>>

CVE-2026-42530

[K000161616: NGINX ngx_http_v3_module vulnerability](#) — HTTP/3 QPACK allocates the session `insert_buffer` from a unidirectional encoder stream's pool. Closing that stream frees the pool while the session keeps a dangling pointer, so a later encoder stream triggers a heap use-after-free in the NGINX worker.

CVE-2026-42533

[K000162097: NGINX map directive and regex matching vulnerability](#) — `map` directive regex matching with capture variables can cause a heap buffer overflow in the NGINX worker process.

The bug is in the stream script engine's two-pass complex-value evaluation. In a vulnerable `stream` configuration using `ssl_preread`, attacker-controlled TLS SNI is fed into regex-backed variables. NGINX first computes the output buffer length, then copies the output. A `map` regex can update the global capture state between those passes: the length pass accounts for a small value, while the copy pass reads a larger regex capture such as `$1`, causing an out-of-bounds heap write in the worker.

The companion [CVE-2026-42533/ PoC](#) is a full-chain exploit. It uses the stream leak primitive to recover heap and libc bases for ASLR bypass, sprays fake `ngx_pool_cleanup_s` records through HTTP request bodies, and then uses the overflow to corrupt an adjacent NGINX pool cleanup pointer. When the corrupted pool is destroyed, cleanup dispatch is redirected to libc `system()`, executing the supplied command.

